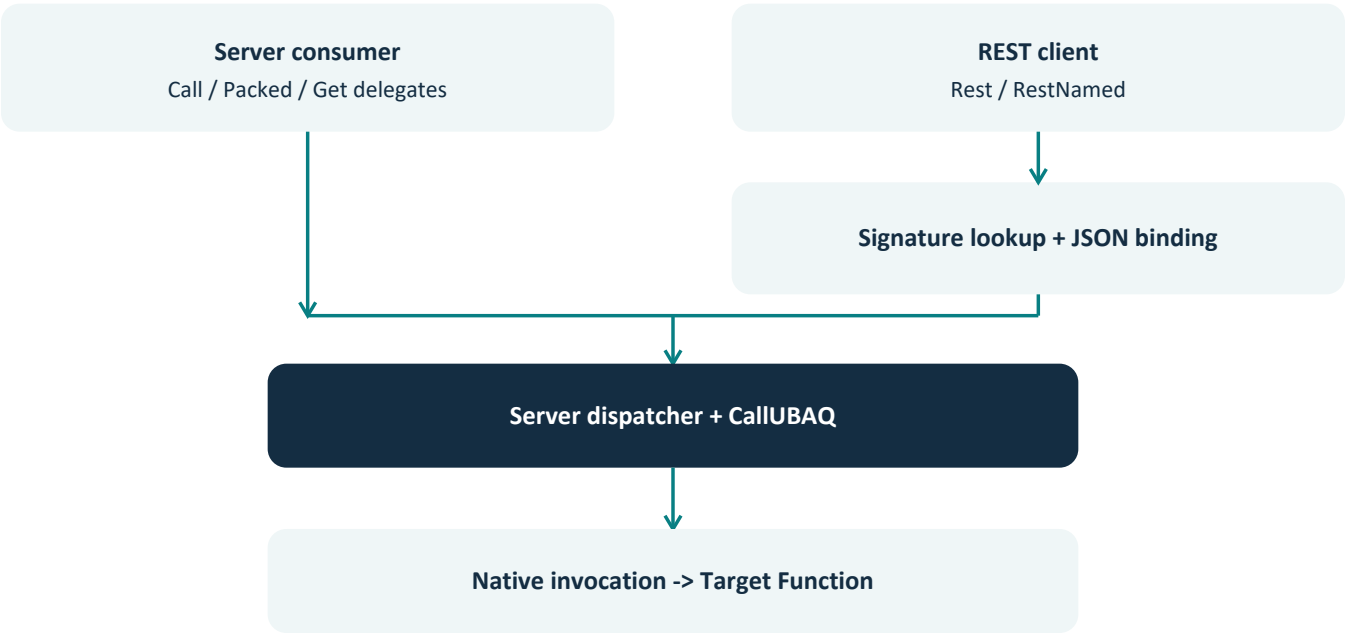
Call

SDK guide

A modern take on Epicor dynamic dispatch.

Server objects. Typed binding. Explicit outcomes.



Returns outputs and details. Native permissions control execution.

Methods contract 1 | Package lanes 4.3.200, 5.1.100, and 5.2.100

October 10, 2026 | Kevin P. Lincecum

Copyright (c) 2026 Kevin P. Lincecum | MIT License

Contents

1. Purpose and terms 3

2. Make a server call 3

3. Select an entry point 4

4. Public API reference 5

5. Server arguments and outputs 6

6. Reuse methods within one request 7

7. Packed arguments 8

8. REST requests 8

9. Error modes and recovery 10

10. Diagnostic reference 11

11. Workload and host constraints 13

12. Installation and compatibility 13

13. Verification status 14

14. Consumer acceptance checklist 14

15. Supplied resources 15

2024.2 port 15

Copyright (c) 2026 Kevin P. Lincecum | MIT License

A modern take on Epicor dynamic dispatch.

Based on FunctionRunner.

Document date: 2026-10-10. Methods contract: 1.

Copyright (c) 2026 Kevin P. Lincecum. Licensed under the MIT License.

This guide uses ASD-STE100-style technical English. It does not claim verification against the official ASD dictionary.

1. Purpose and terms

Call runs an Epicor Function that the caller selects at runtime. The caller supplies the library name, Function name, inputs, and error mode.

Term	Meaning
Call	The product
CallLib	The Epicor Function library that supplies the public API
CallUBAQ	The updatable BAQ that dispatches the request
Target	The Epicor Function that the caller selects
Signature	The declared names, types, and order of the target inputs and outputs
Consumer	Code that uses Call
Dispatch	Entry into the native invocation API
Delegate	A typed .NET method reference
Request	The current server operation and its execution context
Details	The diagnostic DataSet for one call attempt

This SDK consists of native Epicor packages, public Functions, and a source helper. It is not a separate NuGet client library.

2. Make a server call

This example uses an illustrative target named `Pricing.Calculate`. It is not a supplied or verified installed target.

Direction	Position	Name	CLR type
Input	0	iPartNum	System.String
Input	1	iQuantity	System.Decimal
Output	0	oPrice	System.Decimal

Prerequisites:

- Install the matching CallLib and CallUBAQ packages.

- Configure the required native access and company mapping.

- Add a Function-library reference to CallLib in the consumer library.

- Use the actual target signature for production code.

```
var (success, outputs, details) = EfxLib.CallLib.Call("Pricing", "Calculate", new object[] { "PART-123", 10m
    }, "Throw");
decimal price = (decimal)outputs[0];
```

The `m` suffix creates a Decimal value. The output cast matches the illustrative target signature.

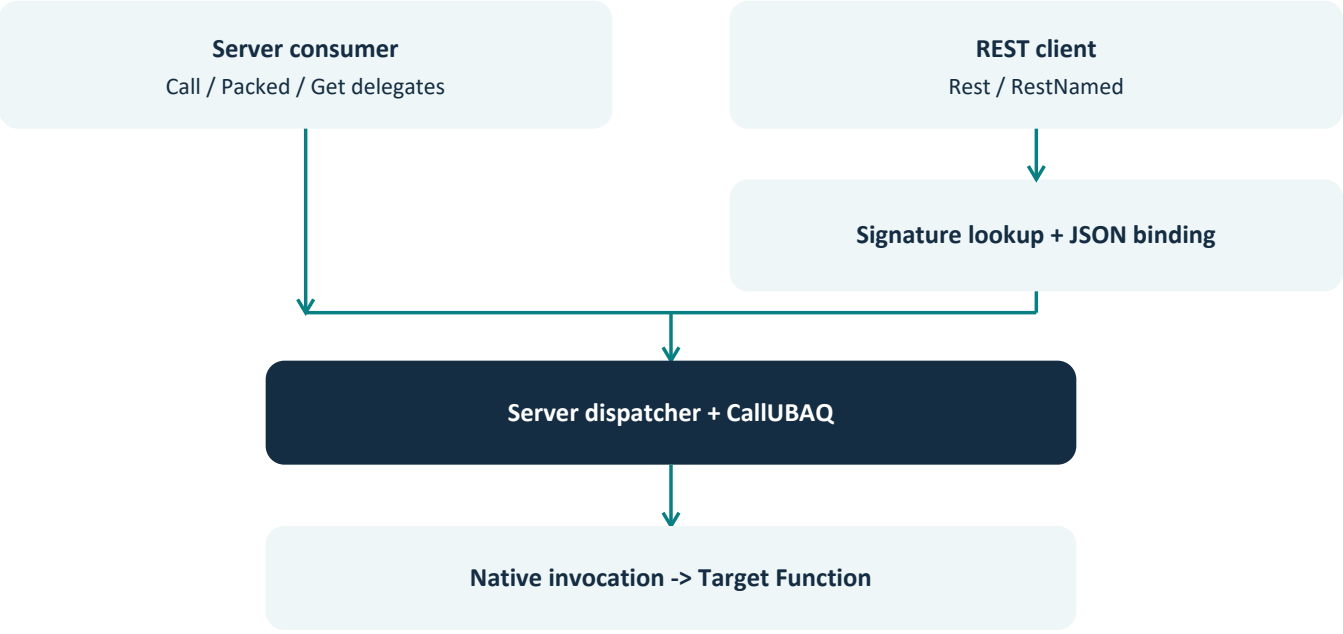
In Throw mode, a normal return indicates success. A failure raises an Epicor business exception.

3. Select an entry point

Requirement	Entry point	Input representation
One server call	CallLib.Call	Ordered object array
Repeated calls in one request	CallLib.Get, then the UseCall helper	Typed Run and Packed delegates
One server call with a scalar or tuple	CallLib.Packed	Object, tuple, or object array
REST call with ordered inputs	CallLib.Rest	JSON array inside a string
REST call with named inputs	CallLib.RestNamed	JSON object inside a string

Copyright (c) 2026 Kevin P. Lincecum | MIT License

`RestCore` and `ReadSignature` are private implementation Functions. Do not use them as consumer entry points.



Returns outputs and details. Native permissions control execution.

Execution paths: shared dispatch with native access control.

Both public REST methods use the same server dispatcher after JSON binding. Native Epicor permissions still control execution.

4. Public API reference

4.1 Call and Packed

Input	Call type	Packed type	Requirement
iLibrary	String	String	Target library name, not blank
iFunction	String	String	Target Function name, not blank
iArguments	Object[]	Object	Target inputs
iErrorMode	String	String	Exactly Throw or Return

Output	CLR type	Meaning
oSuccess	System.Boolean	True when Call accepts the completed response as successful
oOutputs	System.Object[]	Target outputs in signature order
oDetails	System.Data.DataSet	One Call table with one diagnostic row

Copyright (c) 2026 Kevin P. Lincecum | MIT License

The examples assign local names through tuple deconstruction. These local names do not change the native output names.

4.2 Get

`Get` accepts `iErrorMode` as a String. It returns `oMethods` as a DataSet.

The DataSet contains one `Methods` table with one row.

Column	Column type	Value
Version	System.Int32	1
Run	System.Object	Typed delegate with library, Function, and Object[] inputs
Packed	System.Object	Typed delegate with library, Function, and Object inputs

Both delegates return this tuple:

```
(bool Success, object[] Outputs, System.Data.DataSet Details)
```

The complete delegate types are:

```
Func<string, string, object[],
    (bool Success, object[] Outputs, System.Data.DataSet Details)>

Func<string, string, object,
    (bool Success, object[] Outputs, System.Data.DataSet Details)>
```

4.3 Rest and RestNamed

Both Functions accept four String inputs:

Input	Meaning
iLibrary	Target library name
iFunction	Target Function name
iArguments	A string that contains JSON
iErrorMode	Exactly Throw or Return

Both Functions return `oSuccess`, `oOutputs`, and `oDetails`. REST `oOutputs` is a DataSet, not the server Object[] result.

See REST requests for the request and response structures.

5. Server arguments and outputs

The Call and Run argument array must not be null. Each element corresponds to one target input.

Copyright (c) 2026 Kevin P. Lincecum | MIT License

Intended inputs	iArguments for Call or Run
No inputs	`Array.Empty<object>()`
One null input	new object[] { null }
One DataSet	new object[] { dataSet }
One array	new object[] { someArray }
Two values	new object[] { first, second }

Use the target's declared input order and CLR types. The server path does not convert arguments through JSON.

Call passes the actual server objects. Call does not create an isolated copy of those objects.

```
var (success, outputs, details) = EfxLib.CallLib.Call("MyLibrary", "Process", new object[] { myTableset },
    "Throw");
```

`MyLibrary.Process` and `myTableset` are illustrative. The target must declare the matching tableset type.

Read outputs only after success. Use the target's declared output order and CLR types.

An empty output array can indicate a successful target with no outputs. An empty array alone does not prove success.

6. Reuse methods within one request

1. Copy UseCall.cs into the consumer Function's code body.

2. Make the System and System.Data namespaces available to that code.

3. Add the native Function-library reference to CallLib.

4. Acquire the methods with the required error mode.

5. Execute the target through Run or Packed.

`UseCall` is a supplied local helper. Epicor does not automatically provide this helper.

The helper checks the table structure, version, and delegate types. The helper rejects an incompatible carrier with an exception.

The following code assumes that the helper is present:

```
var call = UseCall(() => EfxLib.CallLib.Get("Return"));
var result = call.Run("Pricing", "Calculate", new object[] { "PART-123", 10m });
if (!result.Success)
{
    string message = Convert.ToString(result.Details.Tables["Call"].Rows[0]["Message"]);
    // Apply the application's failure policy here.
    return;
}
decimal price = (decimal)result.Outputs[0];
```

Copyright (c) 2026 Kevin P. Lincecum | MIT License

Both methods retain the error mode from Get. Each execution creates its own result and details.

Keep these delegates within the current server request. Do not retain them in static storage or caches across requests.

Do not serialize delegates or return them through REST. The API does not establish a concurrency guarantee for shared delegates.

7. Packed arguments

Packed converts its input into the same ordered argument array that Run uses.

Supplied value	Target inputs
Bare null	Zero inputs
Object[]	Array elements become individual inputs
An ITuple value	Tuple elements become individual inputs
Another object	One input

Use an outer Object[] when an array or tuple must remain one target input.

```
var (success, outputs, details) = EfxLib.CallLib.Packed("Pricing", "Calculate", ("PART-123", 10m), "Throw");
decimal price = (decimal)outputs[0];
```

The acquired Packed delegate uses the same conversion rules:

```
var call = UseCall(() => EfxLib.CallLib.Get("Throw"));
var result = call.Packed("Pricing", "Calculate", ("PART-123", 10m));
decimal price = (decimal)result.Outputs[0];
```

Use `new object[] { null }` for one null input. Bare null has a different meaning in Packed.

8. REST requests

8.1 Connection requirements

Use the standard authenticated Epicor Function endpoint. Use the authentication and API-key requirements configured for your Epicor installation.

```
POST {server}/api/v2/efx/{company}/CallLib/Rest
POST {server}/api/v2/efx/{company}/CallLib/RestNamed
Content-Type: application/json
```

`{server}` includes the base URL for the Epicor application. `{company}` identifies the authorized company context.

Configure publication, enabled state, company mapping, and caller permissions for Call and the target. Metadata visibility does not grant execution permission.

Copyright (c) 2026 Kevin P. Lincecum | MIT License

Authentication can fail before Call executes. Transport failures can prevent a response after the target executes.

8.2 Positional inputs

Send an outer request object with these four properties:

```
{
  "iLibrary": "Pricing",
  "iFunction": "Calculate",
  "iArguments": "[\"PART-123\",10]",
  "iErrorMode": "Return"
}
```

`iArguments` is a string that contains a JSON array. It is not an array property in the outer request.

Match the target input count and order. Use the string `"[]"` for a target with no inputs.

8.3 Named inputs

Use RestNamed with the exact target input names:

```
{
  "iLibrary": "Pricing",
  "iFunction": "Calculate",
  "iArguments": "{\"iQuantity\":10,\"iPartNum\":\"PART-123\"}",
  "iErrorMode": "Return"
}
```

Names are case-sensitive. Property order does not affect binding.

Supply every declared input. Optional metadata does not supply default values in this implementation.

Use an explicit JSON null when the declared type permits null. Use the string `"{}"` for a target with no inputs.

Call rejects unknown input names. Call also rejects duplicate JSON property names, including duplicate names inside nested objects.

8.4 Type binding

Call reads current target metadata for each REST call. Call converts each JSON value to its declared input type.

The binder retains the original JSON representation until conversion. It does not first convert Decimal inputs through Double.

Date-shaped strings remain strings when the target declares String. JSON type metadata does not select arbitrary CLR types.

A null value cannot bind to a non-nullable value type. The declared type must be available in the server context.

Generic DataSet JSON follows the host's reconstruction rules. It does not preserve an arbitrary CLR column schema.

Use server calls when you must pass existing server objects directly. Use serializable outputs for REST targets.

8.5 Response structure

The following response is illustrative. Epicor can add its normal response envelope.

Copyright (c) 2026 Kevin P. Lincecum | MIT License

```
{
  "oSuccess": true,
  "oOutputs": {
    "Outputs": [
      { "Index": 0, "Name": "oPrice", "Value": 12.50 }
    ]
  },
  "oDetails": {
    "Call": [
      {
        "Id": "example-call-id",
        "Library": "Pricing",
        "Function": "Calculate",
        "Stage": "Dispatch",
        "Status": "Succeeded",
        "DispatchStarted": true,
        "ElapsedMilliseconds": 5,
        "ErrorCode": "",
        "Message": ""
      }
    ]
  }
}
```

Outputs column	Meaning
Index	Zero-based position in the target output signature
Name	Declared target output name
Value	Serialized target output value

In Return mode, check `oSuccess` before reading output values. HTTP 200 alone does not establish target success.

In Throw mode, a Call failure raises a native business fault. Exception.Data contents are not guaranteed across HTTP serialization.

Check both transport status and the Function result in a REST consumer. Do not assume every error includes a Call details row.

9. Error modes and recovery

Mode	Successful call	Failed call
Throw	Returns outputs and details	Raises Ice.BLException
Return	Returns outputs and details	Returns false and details

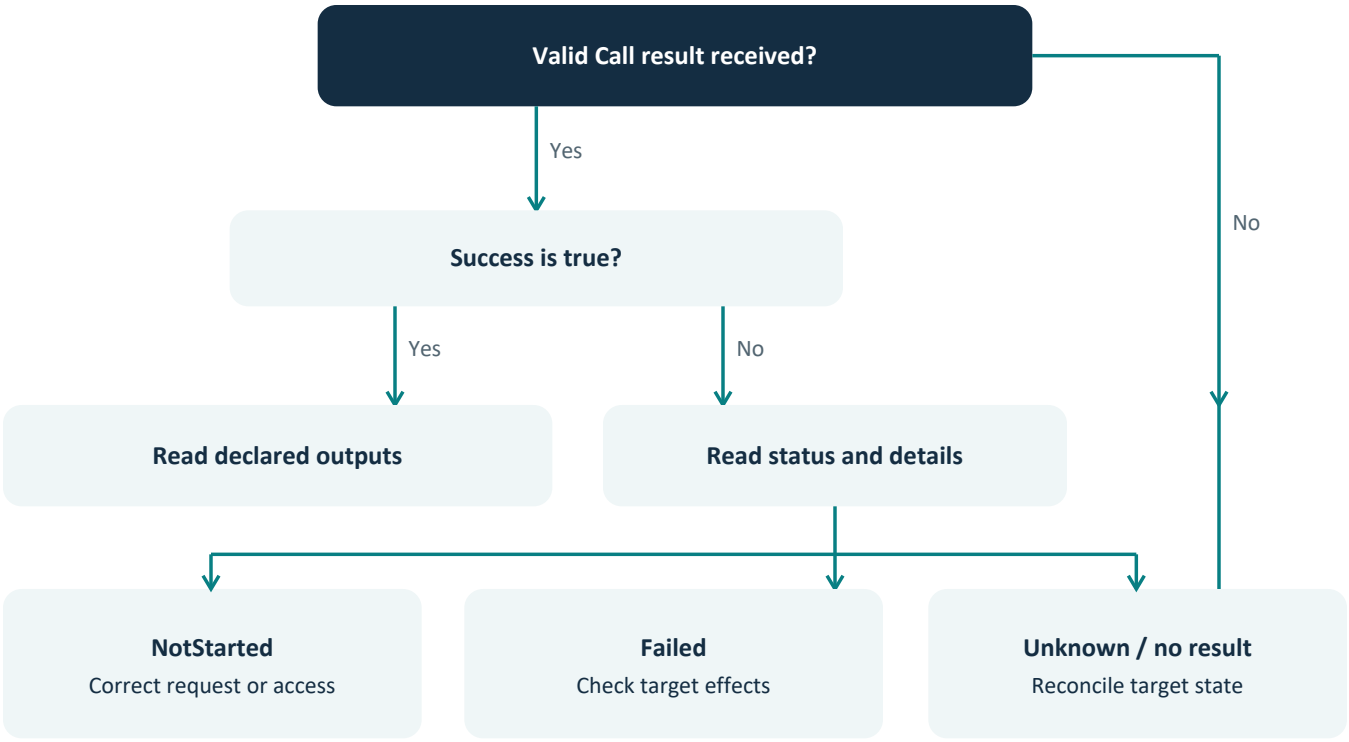
Use the exact mode strings. An invalid mode raises an exception before normal result handling.

Return mode does not suppress authentication failures, transport failures, or helper validation exceptions.

Call does not retry automatically. Call does not guarantee rollback after failure.

Copyright (c) 2026 Kevin P. Lincecum | MIT License

Use this decision sequence:



Recovery: determine target state before repeating an uncertain operation.

For an uncertain outcome:

1. Record the Call Id when available.
2. Read the target's authoritative state.
3. Determine whether the requested operation occurred.
4. Apply the application's recovery policy.
5. Repeat the call only when the application can safely repeat the operation.

Call does not supply an idempotency key or a transaction recovery service. The application owns these policies.

10. Diagnostic reference

Server results contain `Details.Tables["Call"]`. REST results contain `oDetails.Call`.

Each details table contains one row.

Copyright (c) 2026 Kevin P. Lincecum | MIT License

Field	CLR type	Meaning
Id	String	Identifier for this attempt
Library	String	Requested library
Function	String	Requested Function
Stage	String	`Validate`, `Lookup`, `Bind`, or `Dispatch`
Status	String	NotStarted, Succeeded, Failed, or Unknown
DispatchStarted	Boolean, permits DBNull	Whether Call observed entry into native invocation
ElapsedMilliseconds	Int64	Measured duration of this attempt
ErrorCode	String	Error classification
Message	String	Diagnostic explanation

`DispatchStarted=true` does not prove that the target body started or committed. DBNull means that Call cannot establish dispatch entry reliably.

The server path does not use the REST Lookup and Bind stages.

Status	Interpretation
NotStarted	Call reports failure before target dispatch
Succeeded	Call accepted a completed successful response
Failed	Call reports a target failure or an invalid completed result
Unknown	Call cannot establish a complete outcome

Copyright (c) 2026 Kevin P. Lincecum | MIT License

Error code	Typical cause	Consumer action
InvalidRequest	Blank target, null canonical array, or invalid JSON request	Correct the request
SignatureUnavailable	Target metadata cannot supply the signature	Check target identity and metadata access
InvalidArgument	Count, name, or declared-type conversion fails	Compare inputs with the target signature
TargetFailed	Native target invocation raises an error	Check the message and target effects
BridgeUnavailable	CallUBAQ does not acknowledge completion correctly	Inspect CallUBAQ and reconcile target state
OutcomeUnknown	Transport or response failure prevents a reliable outcome	Reconcile target state
InvalidResponse	REST output count differs from the current signature	Check the signature and target effects

Details does not deliberately capture input values, output values, execution tables, or stack traces. Target error messages can contain application information.

Message preserves the reported exception message. It does not include the complete exception object or stack trace.

11. Workload and host constraints

Call does not impose a separate argument-text size cap, input-count cap, or small JSON nesting cap.

The JSON parsers use the largest depth value representable by their Int32 configuration field. This does not guarantee unlimited nesting.

Available memory, CLR type conversion, Epicor request processing, and server configuration still constrain workloads. A tested workload is not a supported maximum.

Call checks duplicate JSON properties with an explicit work stack. Target parameter count, names, order, and types must still match the signature.

The signature query has no row cap. Diagnostic messages are not truncated by Call.

12. Installation and compatibility

Call requires native DynamicQuery, Newtonsoft.Json, and System.Text.Json. Call has no custom Function-library dependency.

Use both packages from the same version directory. Retain the corresponding manifest and hashes.

Copyright (c) 2026 Kevin P. Lincecum | MIT License

Epicor/ICE package version	Package directory
4.3.200	dist/4.3.200
5.1.100	dist/5.1.100
5.2.100	dist/5.2.100

These numbers identify package versions. They do not establish a mapping to an Epicor marketing release.

Follow Install Call for import, native compilation, company mapping, publication, and acceptance checks.

The public packages use sample company CALLDEMO and owner KLINCECUM. Import directly, then configure company mappings, ownership, and security.

Check the CallUBAQ directive independently after import. An overall successful BAQ import can omit the dispatch directive.

Call uses DynamicQuery for metadata and dispatch. It does not call the EFX business object or a Designer service at runtime.

13. Verification status

Local checks cover all three package lanes for source inclusion, package structure, stable hashes, and simulated behavior.

An isolated 5.2.100 native test verified dummy-company import, company reassignment, ownership, company mapping, directive compilation, and successful REST dispatch.

This edition removes inherited argument-size, input-count, JSON-depth, and diagnostic-message caps. Local regression checks cover workloads beyond the former caps.

The packages preserve required XML namespace bindings and consistent BPM identities. The UBAQ workflow omits the malformed empty references collection.

The native test used isolated artifact identities. It did not repeat native 5.1.100 import, ordinary-user regression, or exact raw HTTP Decimal acceptance.

See release status for the precise review boundary. Follow the consumer checklist for the destination installation.

14. Consumer acceptance checklist

1. Check the installed package version against the destination version.

2. Check the installed CallLib and CallUBAQ pair.

3. Execute a harmless target with known inputs and outputs.

4. Check successful outputs against the declared CLR types.

5. Exercise a controlled failure with the selected error mode.

6. Check the details row and application failure handling.

Copyright (c) 2026 Kevin P. Lincecum | MIT License

7. Check REST serialization when the consumer uses REST.

8. Repeat acceptance with the intended user's permissions.

9. Check the application's recovery process for an uncertain outcome.

Use owned synthetic targets for acceptance tests. Do not repeat a business operation merely to test error handling.

15. Supplied resources

- UseCall helper

- Installation guide

- AI integration guide

- Package selection

- Release status

Call Workbench documentation is outside this guide's scope.

2024.2 port

The 4.3.200 lane uses the same maintained source. It passed 61 local checks, 21 native server test groups and four REST cases, with native tests repeated after restart. See 2024 qualification for the installation requirements and exact-package evidence boundary.

Copyright (c) 2026 Kevin P. Lincecum | MIT License

MIT License

Copyright (c) 2026 Kevin P. Lincecum

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.